DATABASE SYSTEMS PROJECT / TECHNICAL DOCUMENTATION

Hostel Room & Complaint Management System

Detailed guide to requirements, relational database, application design, workflows, validation and report alignment.

13	4	5	4
relational tables	database views	stored procedures	application roles

Purpose

Explains the problem HRCMS solves; the ER/EER mapping and normalization; every database table and its key data types; rules and workflows; frontend/API/database connectivity; validation evidence; and differences between the implementation and the supplied report.

Implementation snapshot

React + Vite frontend | Node.js + Express API | MySQL 8.4 | JWT sessions | bcrypt password hashing

Prepared 26 October 2026 | Project review, viva preparation and demonstration

1. Project overview and problem identification

HRCMS replaces paper or spreadsheet-based room registers and complaint tracking with a role-based application backed by a relational database.

Separate spreadsheets and paper complaint registers can cause duplicate room allocations, lost requests, unclear responsibility and limited planning information. HRCMS centralizes users, blocks, rooms, student allotments and maintenance complaints. The database is the shared source of truth; each role sees the workflows it needs.

Objectives

- Store users, students, wardens, staff, blocks, rooms and complaints in a normalized schema.
- Prevent rooms exceeding capacity and students holding two active allocations.
- Let students raise and track issues for their current room.
- Route work to staff with the right specialisation in the relevant block.
- Record status and assignment changes as an auditable timeline.
- Give wardens occupancy, overdue work and staff workload information.

In scope

- Accounts and roles; hostel blocks and rooms; room types; room allocation and history.
- Complaint categories, creation, assignment, rejection, progress, resolution, reopening and feedback.
- Dashboards for occupancy, overdue complaints and staff workload.
- Administrator management of accounts, blocks, rooms, room types and complaint categories.

Mess management, hostel fee payments, visitors, leave/outpass workflows, email/SMS notifications and complaint photo attachments are outside the current implementation.

2. Requirement analysis and roles

Functional requirements describe system behavior; non-functional requirements address security, integrity, performance, usability and maintainability.

ID	Requirement summary	Role / enforcement
FR1	Sign in with email/password; role-scoped access	All / API auth
FR2	Create, edit, deactivate master data and accounts	Administrator
FR3	View occupancy, capacity and vacancies	Warden / views
FR4	Allocate/vacate; check capacity, maintenance, gender	Warden / API + SQL
FR5	At most one active room per student; keep history	Generated unique key
FR6	Raise complaint against current room	Student / API + DB
FR7	Reject duplicate active room/category complaints	Transaction + DB guard
FR8	Assign matching block/specialty worker or reject	Warden / API + DB
FR9	Move assigned complaint through progress/resolution	Staff / state rules
FR10	Rate resolved complaint, close or reopen	Student / API + trigger
FR11	Keep old/new status, actor, remark, time	History trigger
FR12	Show SLA overdue work and staff workload	Warden / views

Role responsibilities

Role	Typical actions
Student	View room, roommates, allocation history; raise complaints; follow timeline; reopen resolved issue; submit feedback; manage profile.
Warden	Manage one block; inspect occupancy; allocate/vacate rooms; assign/reject complaints; view workload and overdue reports.
Maintenance staff	See assigned jobs; start work; resolve with remark; inspect history; manage profile.
Administrator	Manage users, blocks, rooms, room types and categories; view global reports.

Non-functional goals: parameterized SQL, password hashing, role/ownership checks, database constraints, indexed dashboard queries, clear UI, and separation of frontend, API and database.

3. EER model: identity and hostel

USER is the common superclass. STUDENT, WARDEN, MAINTENANCE_STAFF and ADMIN are role subtypes; shared login details are stored once.

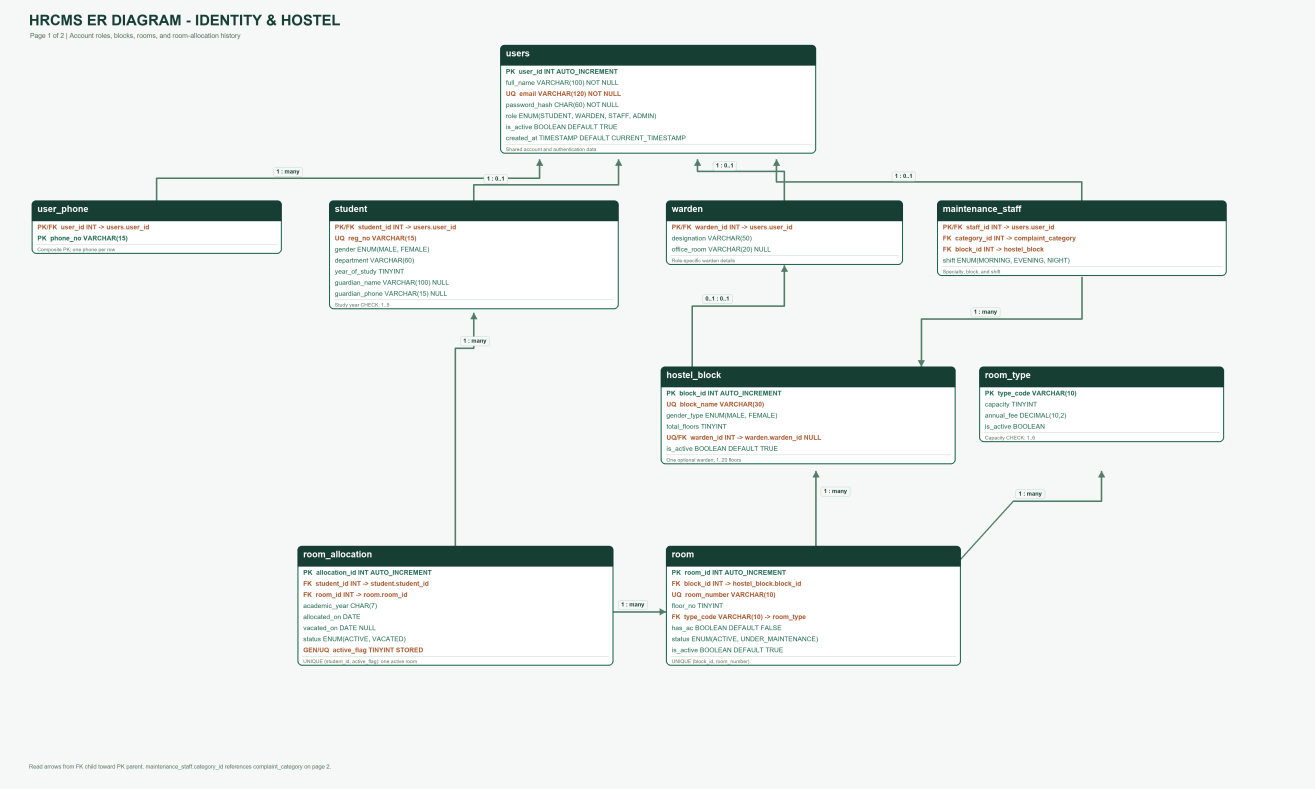


Figure 1. Identity, phone numbers, blocks, room types, rooms and room-allocation relationships.

Relationship interpretation

- A user may have multiple phone rows.
- One block contains many rooms; each room references one room type.
- A block may have one managing warden; a warden manages at most one block.
- Students and rooms are many-to-many over time through ROOM_ALLOCATION. That relationship stores year, dates and status.
- USER specialisation is disjoint: one account has one role. Subtype PKs also reference users.user_id.

Occupancy/vacancy figures are derived from capacity and active allocations. They are not stored as editable counters.

4. EER model: complaint workflow

Complaints link a student and room to a category, optionally assigned worker, history log and optional feedback.

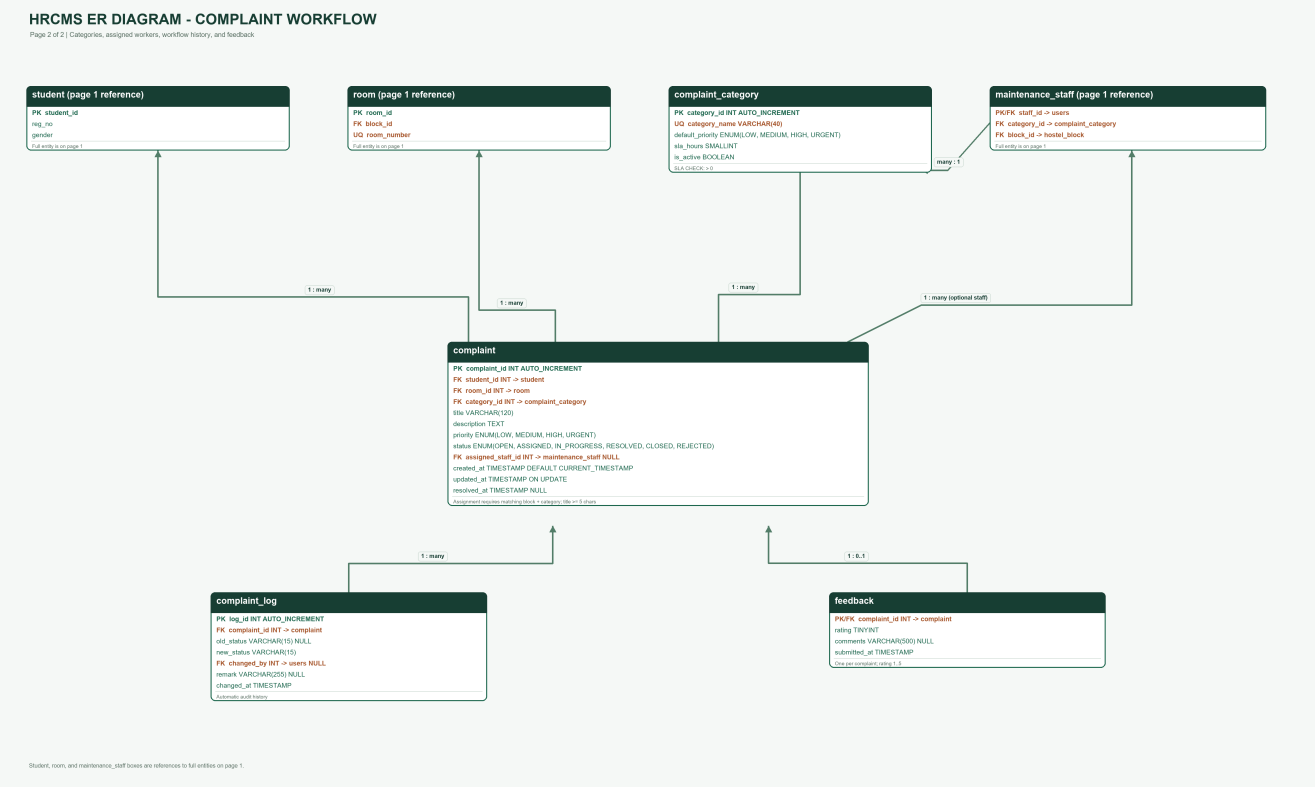


Figure 2. Complaint, category, worker, history and feedback relationships.

Life cycle

OPEN -> ASSIGNED -> IN_PROGRESS -> RESOLVED -> CLOSED after student feedback. A warden may reject an eligible complaint. A student may reopen a resolved issue; assignment is cleared and reopening is logged.

The current room is used when a complaint is submitted. API/database checks ensure the assigned worker matches both category and block. History is written automatically when status or assignment changes.

5. ER/EER-to-relational mapping and normalization

The conceptual model becomes relations by mapping entities, subtype rules, cardinalities, weak entities and multivalued attributes.

ER construct	Relational mapping
Strong entities	Tables with primary keys: users, hostel_block, room_type, complaint_category, complaint.
Role specialisation	Shared fields in users; student, warden and maintenance_staff subtype tables use PKs that are also FKs. ADMIN is a users row and role.
Multivalued phone	user_phone with composite PK (user_id, phone_no), one value per row.
1:N	FK on the many side, such as room.block_id and complaint.student_id.
M:N over time	room_allocation with student and room FKs plus relationship attributes.
Optional 1:1	Unique nullable warden_id in hostel_block.
Weak entities	complaint_log references complaint; feedback uses complaint_id as PK and FK.
Derived facts	Views calculate occupancy/vacancy/age instead of storing duplicated totals.

Normalization

1NF: Each field stores one value. Multiple phone numbers are separate user_phone rows, not a list in one cell.

2NF: Non-key values depend on the complete key. For composite-key user_phone there are no non-key fields dependent on only part of the key. Allocation attributes are kept in room_allocation.

3NF: Facts are stored with the entity they describe. Capacity and fee are in room_type; block gender is in hostel_block; SLA is in complaint_category. This removes transitive dependencies in a flat paper register.

The supplied report also claims BCNF. The key-based structure is consistent with that claim; a formal proof should enumerate each functional dependencies for each relation.

6. Relational schema: identities and hostel

Data types and column names follow database/01_schema.sql. PK = primary key; FK = foreign key; UQ = unique.

users

Shared identity, login and active status for all roles.

Column	MySQL type	Key / rule
user_id	INT AUTO_INCREMENT	PK
full_name	VARCHAR(100)	NOT NULL
email	VARCHAR(120)	NOT NULL, UQ
password_hash	CHAR(60)	bcrypt hash
role	ENUM('STUDENT','WARDEN','STAFF','ADMIN')	NOT NULL
is_active	BOOLEAN	DEFAULT TRUE
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

user_phone

One row per phone; composite key permits multiple values without repeating fields.

Column	MySQL type	Key / rule
user_id	INT	PK/FK -> users.user_id
phone_no	VARCHAR(15)	PK; format CHECK

student

Student-specific profile; account name/email remain in users.

Column	MySQL type	Key / rule
student_id	INT	PK/FK -> users.user_id
reg_no	VARCHAR(15)	NOT NULL, UQ
gender	ENUM('MALE','FEMALE')	NOT NULL
department	VARCHAR(60)	NOT NULL
year_of_study	TINYINT	CHECK 1-5
guardian_name	VARCHAR(100)	nullable
guardian_phone	VARCHAR(15)	nullable

warden

Warden-specific details.

Column	MySQL type	Key / rule
warden_id	INT	PK/FK -> users.user_id
designation	VARCHAR(50)	NOT NULL; default Warden
office_room	VARCHAR(20)	nullable

7. Relational schema: rooms and allocations

Allocation history is preserved; a generated active_flag prevents a student from having multiple active room rows.

hostel_block

Block master; unique nullable warden_id implements at most one warden per block and vice versa.

Column	MySQL type	Key / rule
block_id	INT AUTO_INCREMENT	PK
block_name	VARCHAR(30)	NOT NULL, UQ
gender_type	ENUM('MALE','FEMALE')	NOT NULL
total_floors	TINYINT	CHECK 1-20
warden_id	INT	UQ; nullable FK -> warden
is_active	BOOLEAN	DEFAULT TRUE

room_type

Reusable room classes define capacity and fee.

Column	MySQL type	Key / rule
type_code	VARCHAR(10)	PK
capacity	TINYINT	CHECK 1-6
annual_fee	DECIMAL(10,2)	CHECK >= 0
is_active	BOOLEAN	DEFAULT TRUE

room

Room number is unique within a block. Availability derives from status and occupancy.

Column	MySQL type	Key / rule
room_id	INT AUTO_INCREMENT	PK
block_id	INT	FK -> hostel_block
room_number	VARCHAR(10)	unique with block_id
floor_no	TINYINT	CHECK >= 0
type_code	VARCHAR(10)	FK -> room_type
has_ac	BOOLEAN	DEFAULT FALSE
status	ENUM('ACTIVE','UNDER_MAINTENANCE')	DEFAULT ACTIVE
is_active	BOOLEAN	DEFAULT TRUE

Allocation and complaint-specialty references

room_allocation

Current and past allotments. Index on room_id/status supports occupancy checks.

Column	MySQL type	Key / rule
allocation_id	INT AUTO_INCREMENT	PK
student_id	INT	FK -> student
room_id	INT	FK -> room
academic_year	CHAR(7)	YYYY-YY CHECK
allocated_on	DATE	NOT NULL
vacated_on	DATE	nullable; not before allocation
status	ENUM('ACTIVE','VACATED')	DEFAULT ACTIVE
active_flag	TINYINT GENERATED STORED	1 when active; unique with student_id

Allocation checks

- Student has no other active room.
- Room and block are active; room is not under maintenance.
- Student gender matches the block.
- Room has a free bed according to room_type.capacity.
- Vacating updates state/date and retains the historical row.

complaint_category

Maintenance speciality, default priority and service target.

Column	MySQL type	Key / rule
category_id	INT AUTO_INCREMENT	PK
category_name	VARCHAR(40)	NOT NULL, UQ
default_priority	ENUM('LOW','MEDIUM','HIGH','URGENT')	DEFAULT MEDIUM
sla_hours	SMALLINT	CHECK > 0
is_active	BOOLEAN	DEFAULT TRUE

maintenance_staff

Worker speciality and block govern assignment eligibility.

Column	MySQL type	Key / rule
staff_id	INT	PK/FK -> users.user_id
category_id	INT	FK -> complaint_category
block_id	INT	FK -> hostel_block
shift	ENUM('MORNING','EVENING','NIGHT')	DEFAULT MORNING

8. Relational schema: complaints and feedback

Current complaint state is separate from audit events and feedback.

complaint

Current workflow state and links to student, room, category and optional worker.

Column	MySQL type	Key / rule
complaint_id	INT AUTO_INCREMENT	PK
student_id	INT	FK -> student
room_id	INT	FK -> room
category_id	INT	FK -> complaint_category
title	VARCHAR(120)	CHECK length >= 5
description	TEXT	NOT NULL
priority	ENUM('LOW','MEDIUM','HIGH','URGENT')	NOT NULL
status	ENUM('OPEN','ASSIGNED','IN_PROGRESS','RESOLVED','CLOSED','REJECTED')	DEFAULT OPEN
assigned_staff_id	INT	nullable FK -> maintenance_staff
created_at/updated_at	TIMESTAMP	automatic
resolved_at	TIMESTAMP	nullable

complaint_log

Audit timeline inserted by triggers for status/assignment changes.

Column	MySQL type	Key / rule
log_id	INT AUTO_INCREMENT	PK
complaint_id	INT	FK -> complaint
old_status	VARCHAR(15)	nullable
new_status	VARCHAR(15)	NOT NULL
changed_by	INT	nullable FK -> users
remark	VARCHAR(255)	nullable
changed_at	TIMESTAMP	default current time

feedback

One rating/comment per resolved complaint; feedback closes it.

Column	MySQL type	Key / rule
complaint_id	INT	PK/FK -> complaint
rating	TINYINT	CHECK 1-5
comments	VARCHAR(500)	nullable
submitted_at	TIMESTAMP	default current time

There are 13 physical tables; views are query objects, not extra tables.

9. DDL, integrity rules and database logic

Keys and constraints protect relationships; triggers and procedures enforce rules during changes.

Object	Count / names	Purpose
Tables	13	Persistent data; DDL in database/01_schema.sql.
Views	4: room occupancy, complaint dashboard, overdue complaints, staff workload	Joined/derived dashboard and reporting data.
Procedures	5: allocate, vacate, raise complaint, assign complaint, update status	Controlled operations and business checks.
Triggers	15 in current implementation	Validation, history logging and feedback closure.
Constraints	PK, FK, unique, CHECK, ENUM, indexes	Prevent invalid references, duplicates and out-of-range data.

Core business rules

- Room capacity, gender, maintenance and active allocation are checked before allocation.
- Generated active_flag plus a unique key enforces one active room per student.
- Students can complain only about their active room; duplicate open room/category complaints are refused.
- Assignment requires authorized warden/admin and same-block matching specialty staff.
- Invalid status jumps such as OPEN straight to RESOLVED are rejected.
- Only the student owner can rate their resolved complaint; feedback closes it.
- Triggers record actor, status change and remark; API sets actor context.

Setup scripts do not DROP databases or tables; setup expects an empty database. Authentication and record ownership checks are performed by the API.

10. Business logic and database connectivity

The browser calls the Express API over HTTP; the API connects to MySQL through a pooled driver. The frontend never connects directly to the database.

Tier	Technology	Responsibilities
Frontend	React 19 + Vite	Role-based pages, forms, dashboards, filters and timelines. Sends JSON and bearer token.
Backend	Node.js + Express 5	REST routes, login, role/ownership checks, validation, transactions, error handling and connection pool.
Database	MySQL 8.4	Persistent records, constraints, procedures, triggers, views, transaction locks and audit trail.

Complaint request path

Step	What happens
1	Student signs in; API compares password against bcrypt hash and returns signed JWT.
2	Frontend posts category, title, description and optional priority.
3	API validates request, role and ownership; SQL parameters separate values from query syntax.
4	Transaction invokes database operation; MySQL checks room and duplicate rules; triggers write history.
5	API returns result; frontend refreshes. Warden assigns; staff progresses/resolves; student provides feedback.

Transactions/locks prevent simultaneous requests from claiming the last bed or creating duplicate open room/category complaints. Deadlocks are retried by the API. MySQL passwords and JWT secrets belong in local backend/.env and must not be included in reports or Git.

11. Frontend design, API and security

The interface and REST API implement role boundaries while database rules preserve data integrity.

Workspace	Screens and actions
Student	Room, roommates, history, create complaints, timeline, reopen, feedback, profile/password.
Warden	Block overview, availability, allocate/vacate, assign/reject complaints, workload and overdue reports, CSV export.
Maintenance staff	Assigned jobs, start, resolve with remark, history, profile/password.
Administrator	Global reports; create/edit/deactivate users, blocks, rooms, types and categories.

Representative endpoints

Area	Routes
Authentication	POST /api/auth/login; GET /api/auth/me; GET/PATCH /api/profile; PATCH /api/profile/password
Complaints	GET/POST /api/complaints; GET /:id; POST /:id/assign; PATCH /:id/status; POST /:id/feedback
Room allocation	GET /api/rooms; GET /api/rooms/my; POST /api/rooms/allocate; POST /api/allocations/:id/vacate
Reports/admin	GET /api/dashboard; GET /api/reports/overdue; /api/admin/:resource

Security controls

- Passwords use bcrypt hashing, not reversible plaintext storage.
- JWT identifies sessions; API checks roles and record ownership.
- Parameterized queries reduce SQL injection risk.
- Helmet security headers and login rate limiting are configured.
- Deactivation preserves records; active allocations/assignments block unsafe changes.
- The frontend is not a security boundary: API/database check sensitive operations.

12. Data population (DML) and demonstration records

The seed script adds sample accounts, hostel structure, allocations and complaints using the same database rules.

DML means adding, updating or querying rows. backend/scripts/seed.js populates the demo dataset. Allocation and complaint mutations go through procedures/triggers so sample records follow the same rules as normal operations.

The checked-in README documents a fresh seed of 24 users, 3 blocks, 4 room types, 13 rooms, 12 students, 8 maintenance staff, 6 categories, 12 historical/current allocations and 10 complaints. Demo accounts and their shared demo password are documented in README; do not use them in production.

A later request asked for 50 students in each block and at least five workers in each block, with one warden per block. That customized scale is not reflected in committed seed documentation. The local database may contain manually added records, but fresh setup will use the smaller sample unless seed is updated.

Viva explanation: “The schema supports many students and staff. The starter dataset is smaller. To reproduce the larger demo after clean setup, extend the seed and verify the per-block counts.”

Setup sequence

- Install Node.js and MySQL.
- Configure backend/.env locally with database connection values and a strong JWT_SECRET.
- Run npm run db:setup against an empty database; scripts do not drop existing data.
- Run npm run db:seed once for sample records.
- Build and start the application using project scripts; use demo accounts only for presentations.

13. Testing, validation and current status

The recorded validation covers the database, API and browser so rule violations are checked at multiple levels.

Layer	Example checks
Database	FK/CHECK behavior; capacity and gender; one active room; duplicate complaints; transitions; history; feedback.
API	Invalid login; missing token; role, ownership and block scope; staff speciality; admin validation.
Browser	Student complaint; warden assignment; staff resolve; feedback closes issue; dashboards and mobile layout.

Recorded results

- Repository validation says npm run check passed, including backend syntax and frontend production build.
- The validation record states 31 integration tests passed against a newly created real MySQL test database.
- Concurrency checks covered the last available bed and duplicate complaint submissions.
- Browser workflow covered student, warden and staff, including resolution and feedback closure.
- The temporary test database was removed; development data was preserved.

These results are from the repository validation record dated 25 October 2026. This PDF summarizes that record; no new test run was performed to create this document.

14. Alignment with the supplied report

The project follows the main design in the report and requirements, and code has progressed beyond some older report sections.

Report area	Current implementation
Problem / requirements	Aligned: room allocation, complaints, accountability and dashboards.
EER and relational mapping	Aligned: same 13-table core, roles, allotment history, complaint log and feedback.
Normalization	Aligned structurally: phones separated; room type/category/block facts separated; occupancy derived.
DDL / DML	Implemented through schema, logic and repeatable seed scripts.
Frontend / backend	React + Vite client and Express REST API with JWT and MySQL connectivity.
Business rules / reports	Constraints, procedures, triggers, views and API authorization.
Administrator CRUD	Implemented beyond report, which labels those routes still to build.
Versions / schema extensions	Code uses MySQL 8.4 and React 19; report describes MySQL 8.0/MariaDB and React 18. Extra is_active fields support deactivation.
Sample data scale	Committed seed does not yet match later 50 students/block request.
Future scope	Email/SMS, photos, mess, visitors, leave and online fee payment remain out of scope.

Viva-ready summary

“HRCMS is a three-tier hostel room and complaint application. It has a 13-table normalized MySQL schema. React communicates with Express, which authenticates users and applies role and ownership checks. MySQL constraints, triggers and procedures protect allocation and complaint rules; views provide occupancy, overdue and workload reports. The implementation follows the report and extends administration and integrity checks. The reproducible seed still needs expansion if we want 50 students per block.”

15. Key terms and source files

Use these definitions in viva answers and these paths to locate the implementation.

Term	Plain-language meaning
Primary key	Column(s) uniquely identifying a row.
Foreign key	Column(s) referencing a valid row in another table.
Unique constraint	Prevents duplicates such as emails or multiple active room rows.
CHECK	Rejects values outside a rule or range.
Trigger	Database code that runs automatically on insert/update.
Stored procedure	Named database operation for controlled work.
View	Saved query for joined or calculated information.
Transaction	Statements committed together or rolled back together.
Normalization	Organizing tables to reduce repeated facts and update anomalies.
JWT	Signed token that identifies an authenticated session.
bcrypt	Password hashing; the stored hash is not reversible to plaintext.

Implementation reference

Purpose	Project file
Schema	database/01_schema.sql
Views/triggers/procedures	database/02_logic.sql; database/03_integrity_guards.sql
Setup / overview	database/README.md; backend/scripts/setup-db.js
Demo data	backend/scripts/seed.js
API	docs/API.md
Scope and setup	docs/SPEC.md; README.md
Validation record	docs/VALIDATION.md
ER diagrams	output/diagrams/hrcms-er-diagram-core.png; hrcms-er-diagram-complaints.png
Assessment source	HRCMS_Project_Report.pdf supplied by user

Prepared from repository implementation and the supplied HRCMS report. Confirm final demo counts before presentation.